Reproduction Instructions

PART ONE: Prepare the Project, Environment, and Data

1. Obtain the Project Code and Navigate to the Project Directory

1.1 Option A: Clone the Repository Using Git

```
git clone https://github.com/mark6880/KoeiNet.git
cd KoeiNet
```

1.2 Option B: Download and Extract the ZIP Archive

Download the ZIP archive from the repository page, extract it, and navigate to the root directory of the extracted KoeiNet project.

2. Create and Activate a Virtual Environment

2.1 Create a Virtual Environment

It is recommended to use Python's built-in venv module to create an independent and isolated environment:

```
python -m venv venv
```

2.2 Activate the Virtual Environment

Windows (PowerShell / CMD):

```
.\venv\Scripts\activate
```

macOS / Linux:

```
source venv/bin/activate
```

3. Install the Required Python Packages

Optionally upgrade pip within the virtual environment:

```
python -m pip install --upgrade pip
```

Install all dependencies listed in requirements.txt:

```
pip install -r requirements.txt
```

4. Install FFmpeg for Video and Audio Processing

FFmpeg is required for video frame extraction and audio processing. Select the appropriate installation method for your operating system.

✓ Windows: Download and extract FFmpeg, and then either place ffmpeg.exe in the repository root directory (KoeiNet/) or add it to the system PATH:

```
https://www.gyan.dev/ffmpeg/builds/
```

✓ macOS: Install FFmpeg using Homebrew:

```
brew install ffmpeg
```

✓ Ubuntu/Debian: Install FFmpeg using the apt package manager:

```
sudo apt update && sudo apt install -y ffmpeg
```

5. Download and Organize the Raw Data

The original panoramic video and corresponding GPS trajectory data used in the Nanluoguxiang use case are available through the following Figshare link:

```
https://figshare.com/s/1c882d0430802b474d8c
```

After downloading the data, create an input/ directory under the KoeiNet project root and place both files in this directory.

> **Academic-use statement:** These data files are provided exclusively for academic research and must not be used for commercial purposes or any other unauthorized purposes.

PART TWO: Run the Code and Generate Continuous Multisensory Street Assessment Outputs

This part covers the core functionality of KoeiNet and presents the visual assessment, emotional assessment, and soundscape assessment results and visualizations generated for the Nanluoguxiang use case.

1. Run Program 01

After confirming that the original video has been placed in the input/ directory, run Program 01:

```
python scripts/run_program_01.py
```

Once the input video has been successfully detected, the terminal will display a processing-start message and real-time progress information.

2. Locate the Output Directory

The default output directory is:

```
output/<VIDEO_NAME>/
```

After processing is completed, multiple subdirectories will be generated under this output directory.

3. Review the Generated Results

KoeiNet generates an integrated set of outputs, including result data files, figures, and interactive visualization interfaces.

3.1 Review the Visual and Emotional Assessment Outputs

The results are written to the stats/ directory, with the following structure:

```
output/<VIDEO_NAME>/stats/
├─ color_analysis/
├─ emotion/
├─ green_view/
├─ people_count/
└─ visual_elements/
```

Using green_view/ as an example, the directory structure is:

```
output/<VIDEO_NAME>/stats/green_view/
├─ green_view_index.csv
├─ green_view_index_chart.png
└─ green_view_index_interactive.html
```

Correspondence to the Nanluoguxiang Use Case: The CSV file contains the complete result data for the Nanluoguxiang use case. The PNG file provides a visualization based on these results, while the HTML file provides an interactive visualization interface. Frame-level results can be examined individually, and summary statistics such as means and standard deviations can be derived through straightforward calculations.

3.2 Review the Soundscape Assessment Outputs

The results are written to the audio_events/ directory, with the following structure:

```
output/<VIDEO_NAME>/audio_events/
├─ <VIDEO_NAME>.wav
├─ audio_events_proportion.csv
├─ ...
└─ top_events.png
```

Correspondence to the Nanluoguxiang Use Case: The CSV file contains the complete result data for the Nanluoguxiang use case, while the PNG file provides a visualization based on these results.

PART THREE: Run the Code and Generate Problem-Episode Identification Outputs (Optional)

This part presents a derived application of KoeiNet. Problem episodes are identified by combining the continuous multisensory street assessment results with human-adjudicated annotations. The repository provides the annotation CSV file used in the Nanluoguxiang use case, as specified below, together with a tutorial for creating custom annotations in the Appendix.

1. Prepare the Data

Confirm that the required Program 01 output directory is available at:

```
output/<VIDEO_NAME>/
```

Confirm that the required human-adjudicated annotation CSV file is available at:

```
usecase/validation/final_annotation_labels_adjudicated.csv
```

2. Run Program 02

After confirming that both the Program 01 output directory and the human-adjudicated annotation CSV file exist, run the following command from the project root directory:

```
python scripts/run_program_02.py --program_01_output "output/<VIDEO_NAME>" --annotation_csv "usecase/validation/final_annotation_labels_adjudicated.csv"
```

3. Locate the Problem-Detection Outputs

By default, the results are written to:

```
output/<VIDEO_NAME>/problem_detection/
```

The main output files include:

```
output/<VIDEO_NAME>/problem_detection/
├─ segment_problem_priority.csv
├─ problem_episodes.csv
├─ ...
└─ problem_detection_summary.md
```

Appendix: Generate an Annotation Template, Complete Manual Annotations, and Run Problem Detection Through the GUI

1. Install the Required GUI Dependencies

Install all additional dependencies listed in requirements_gui.txt:

```
pip install -r requirements_gui.txt
```

2. Generate an Annotation Template

```
python scripts/run_program_02.py --program_01_output "<PROGRAM01_OUTPUT_DIR>" --create_annotation_template
```

Example:

```
python scripts/run_program_02.py --program_01_output "output/<VIDEO_NAME>" --create_annotation_template
```

This command generates an annotation-template CSV file under the validation/ directory within the Program 01 output directory.

3. Launch the Program 02 GUI

```
python scripts/run_program_02.py --launch_gui
```

4. Complete GUI Workflow

The following steps must be completed in sequence:

1) Select the Program 01 output directory, for example, output/<VIDEO_NAME>/.
2) When no annotation file is available, click Create Annotation File. Alternatively, the template generated in Step 2 will appear automatically.
3) On the Annotation page, enter or verify the human-scoring fields for each record. Common fields include:
    a) comfort_score (1–5)
    b) vitality_score (1–5)
    c) soundscape_pleasantness (1–5)
    d) soundscape_eventfulness (1–5)
    e) overall_problem_severity (1–5)
    f) confidence_score (1–5)
    g) main_problem_labels (semicolon-separated)
    h) annotator_notes
4) After completing the annotations, click Save Annotation. The file will be saved under output/<VIDEO_NAME>/validation/.
5) Navigate to the Coefficients page, load or retain the default configs/street_type_coefficients.yaml file, adjust the coefficients as needed, and then save or apply the settings.
6) Set the detection parameters in the GUI parameter panel: Top K, Priority Threshold, and Max Gap Seconds. Example settings are Top K = 2, Priority Threshold = 0.4, and Max Gap Seconds = 5.
7) Click Run Problem Detection, wait for processing to complete, and review the logs and progress information.